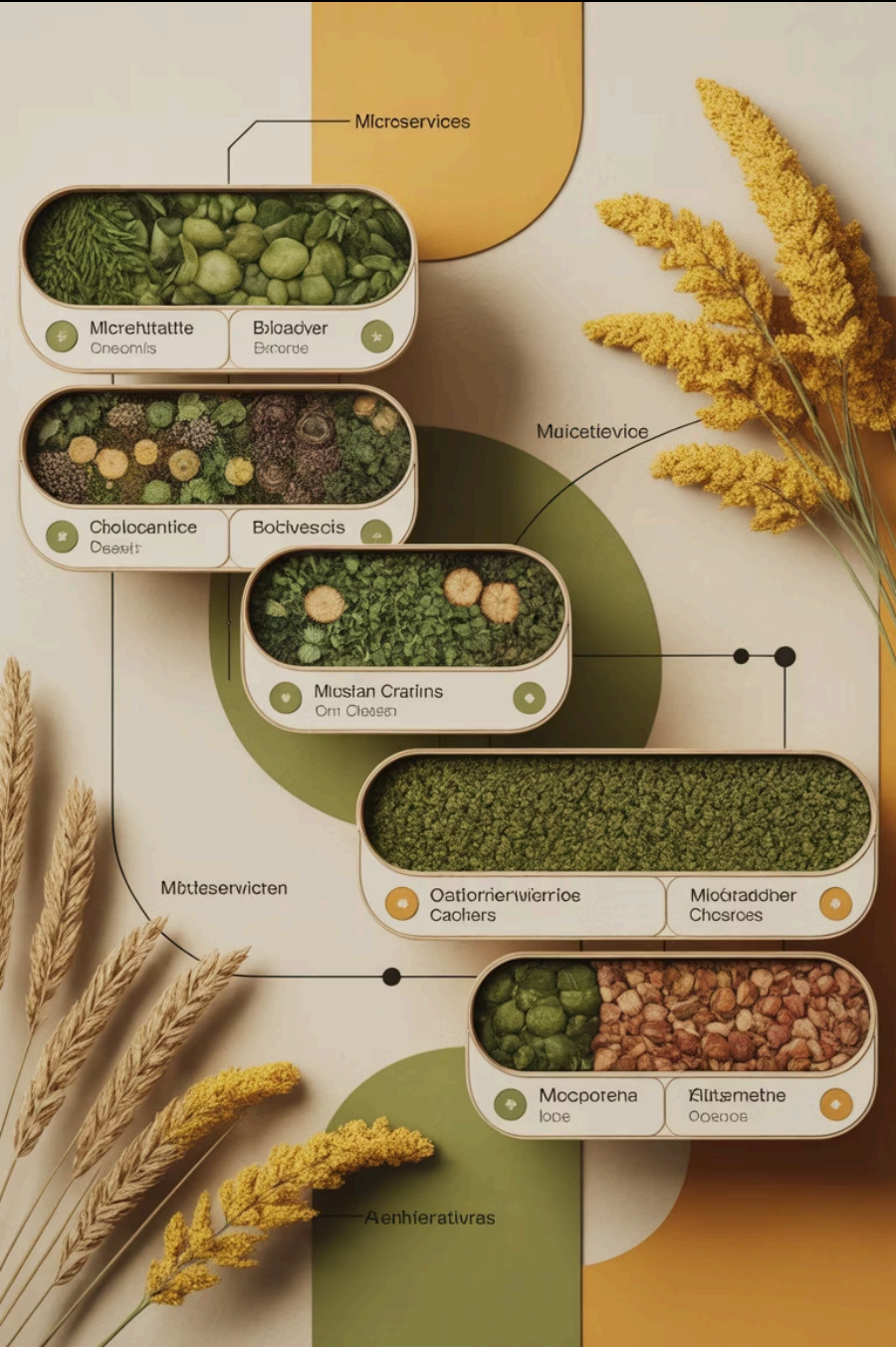




Migração Gradual para Microserviços

Documentação da Transição

Este documento descreve o estado atual do projeto monolítico, aponta pontos positivos e negativos da abordagem atual, explica a separação dos serviços auth, users e events em projetos isolados, e fundamenta os benefícios e riscos da refatoração.

Visão Geral do Projeto Atual

Estrutura Principal

O projeto está organizado em **NestJS** com estrutura modular típica contendo módulos, controllers, services, DTOs, pipes e guards.

src/ contém o código-fonte principal com módulos para auth, users, events, common, database e arquivos de bootstrap.

test/ inclui testes e2e, factories e mocks para validação completa do sistema.

Tecnologias

- Framework NestJS com arquitetura modular
- Banco de dados gerenciado via migrations
- Testes e2e e factories já implementados
- Docker e docker-compose para infraestrutura
- Scripts de migração e deploy automatizados

Módulos Principais do Sistema



Auth

Controladores, serviços, DTOs, entidades e estratégias relacionadas à autenticação com JWT, refresh tokens e fluxos de login/logout



Users

Endpoints, serviços e entidades de usuário para gerenciamento completo do cadastro e perfis



Events

Lógica e endpoints que manipulam eventos do domínio com DTOs, entidades e serviços especializados



Common

Guardas, interceptors, validadores, decorators e utilitários compartilhados entre módulos

Pontos Positivos da Arquitetura Monolítica

A estrutura atual oferece vantagens significativas que facilitam o desenvolvimento e manutenção inicial do projeto.



Simplicidade

Um único repositório, configuração e pipeline de CI/CD simplificam o onboarding de novos desenvolvedores e reduzem a curva de aprendizado inicial.



Testes Integrados

Testes e2e cobrem fluxos completos entre módulos, validando a integração de forma eficiente e confiável.



Compartilhamento Fácil

O diretório common/ centralizado evita duplicação imediata de código e facilita a reutilização de componentes entre módulos.



Menor Sobrecarga Operacional

Um único processo e deploy reduzem a complexidade operacional no início do projeto e aceleram o time-to-market.



Coerência Transacional

DataSource único simplifica a evolução do schema sem necessidade de coordenação cross-service e garante consistência transacional.

Limitações do Monólito

Por que migrar para microsserviços?

Escala e Autonomia Limitadas

Times são impactados por deploys cruzados, reduzindo a velocidade de entrega e criando dependências desnecessárias entre equipes.

Ciclo de Deploy Pesado

Pequenas mudanças exigem rebuild e deploy do monólito inteiro, aumentando o tempo de feedback e o risco de cada release.

Acoplamento Tecnológico

Banco de dados compartilhado e common/ centralizado podem virar gargalo, limitando escolhas tecnológicas e escalabilidade específica.

Risco em Mudanças de Contrato

Sem contratos explícitos, alterações internas quebram dependentes de forma imprevisível, dificultando a evolução do sistema.

Políticas Distintas Difíceis

Implementar políticas diferentes por serviço (escalonamento, linguagem, bancos) é praticamente impossível na arquitetura atual.

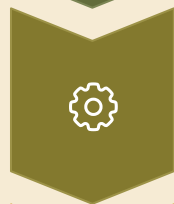
Estratégia de Migração

Separar serviços em projetos **isolados** de forma **incremental**, começando por auth, depois users e finalmente events.



Fase 1

Levantamento, justificativa, plano e materiais didáticos



Fase 2

Execução da migração passo a passo



Validação

Testes de contrato e monitoramento contínuo

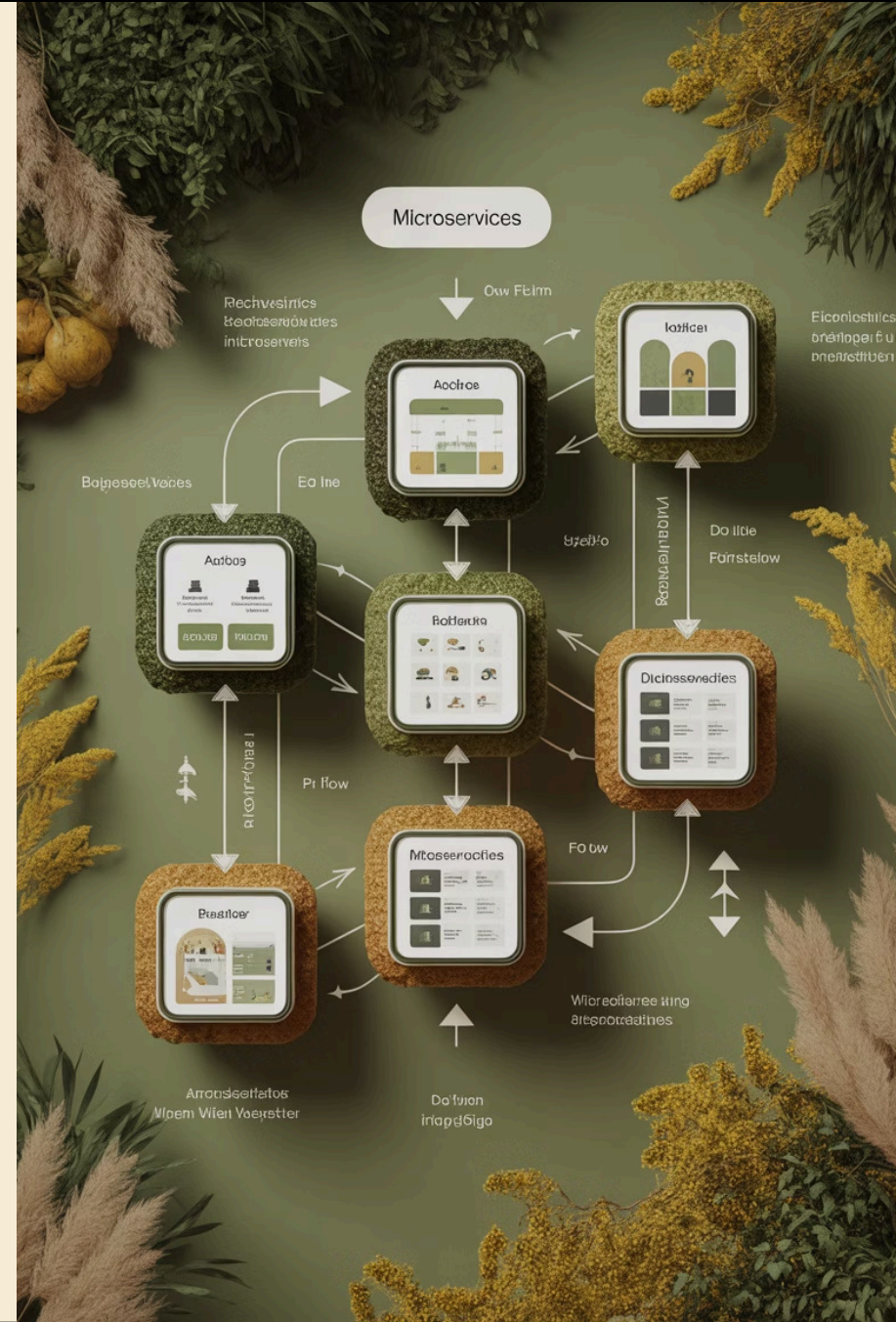
MIGRATION Roadmap

Actito Asibonot Innitip
no ôâôrelhe adôôes
jolofoetteine



ΕΜΑΤΟΟ ΝΑΡΡ

Αεζβηηιό τσλντρελνρελες
ôlorlell ôlmlhla: ôlonεεεο
laleôlôh alnôelheilrhnentle
vôgerlôooloolow



Riscos e Custos da Migração

Overhead Operacional

Mais serviços significam mais infraestrutura, observabilidade, monitoramento e complexidade de gerenciamento.

Complexidade de Dados

Separar schemas exige estratégias como event sourcing, CDC, sincronizações e domain events para manter consistência.

Latência e Resiliência

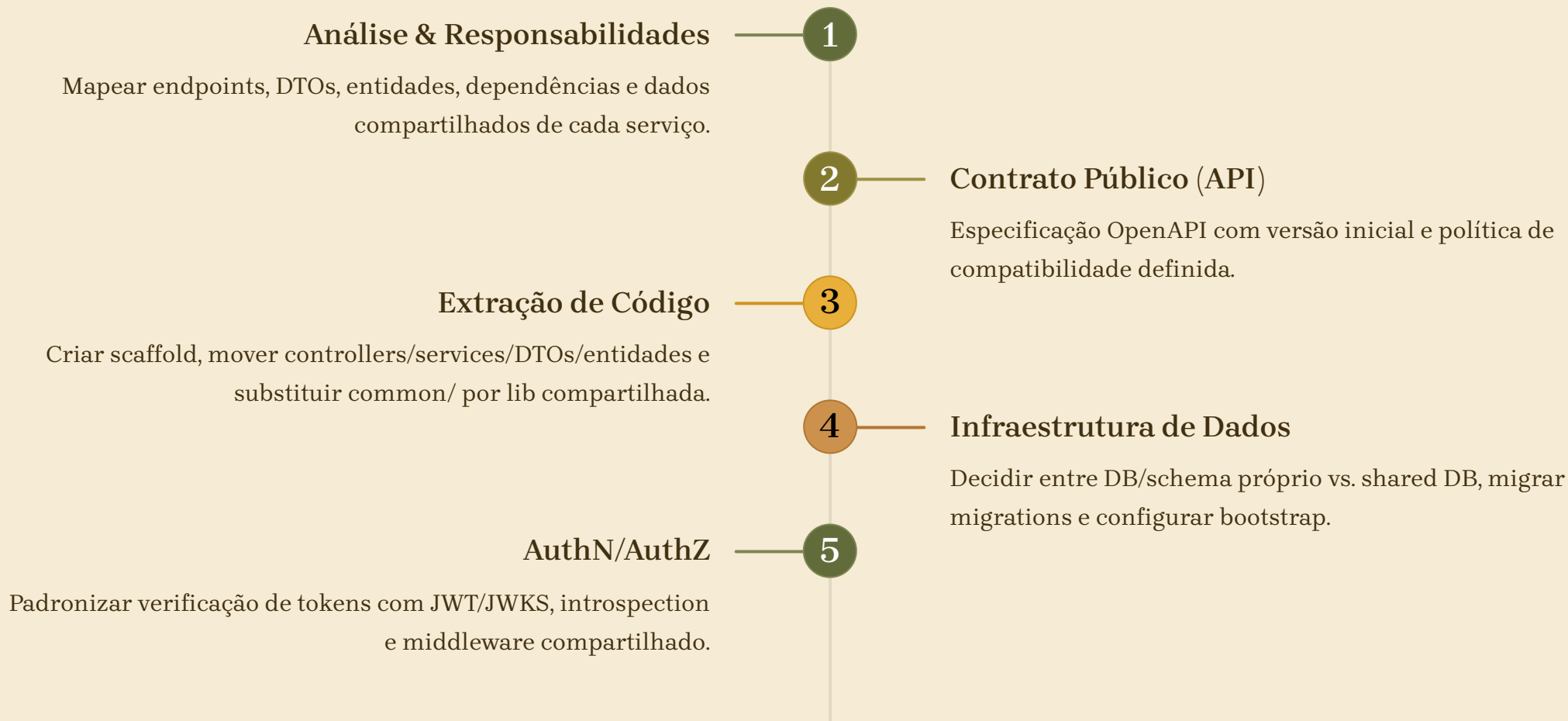
Chamadas inter-serviço introduzem latência e exigem implementação de tolerância a falhas e circuit breakers.

Esforço de Arquitetura

Necessidade de libs compartilhadas, pipelines CI/CD individuais e catálogo de contratos bem documentado.

Processo de Migração Incremental

Ordem recomendada: Auth » Users » Events



Comunicação Entre Serviços

Estratégia Inicial

Começar com **REST** implementando timeouts e retries para garantir resiliência básica.

Avaliar migração para **eventos** usando RabbitMQ ou Kafka especialmente para o serviço de events, permitindo comunicação assíncrona e desacoplamento temporal.



Testes, CI e Deploy

01

Testes & CI

Reaproveitar testes e2e como testes de contrato. Pipeline CI por serviço incluindo build, lint, unit e e2e leves.

03

Cutover & Monitoramento

Estratégia canary ou side-by-side, comutar tráfego gradualmente com monitoramento contínuo.

02

Deploy & Observabilidade

Dockerizar serviços, implementar health checks, logs estruturados, métricas e tracing distribuído.

04

Limpeza

Remover código legado do monólito e transformar common/ em biblioteca versionada.

Contrato Mínimo por Serviço

Input

HTTP JSON com DTOs validados seguindo padrões consistentes de nomenclatura e estrutura.

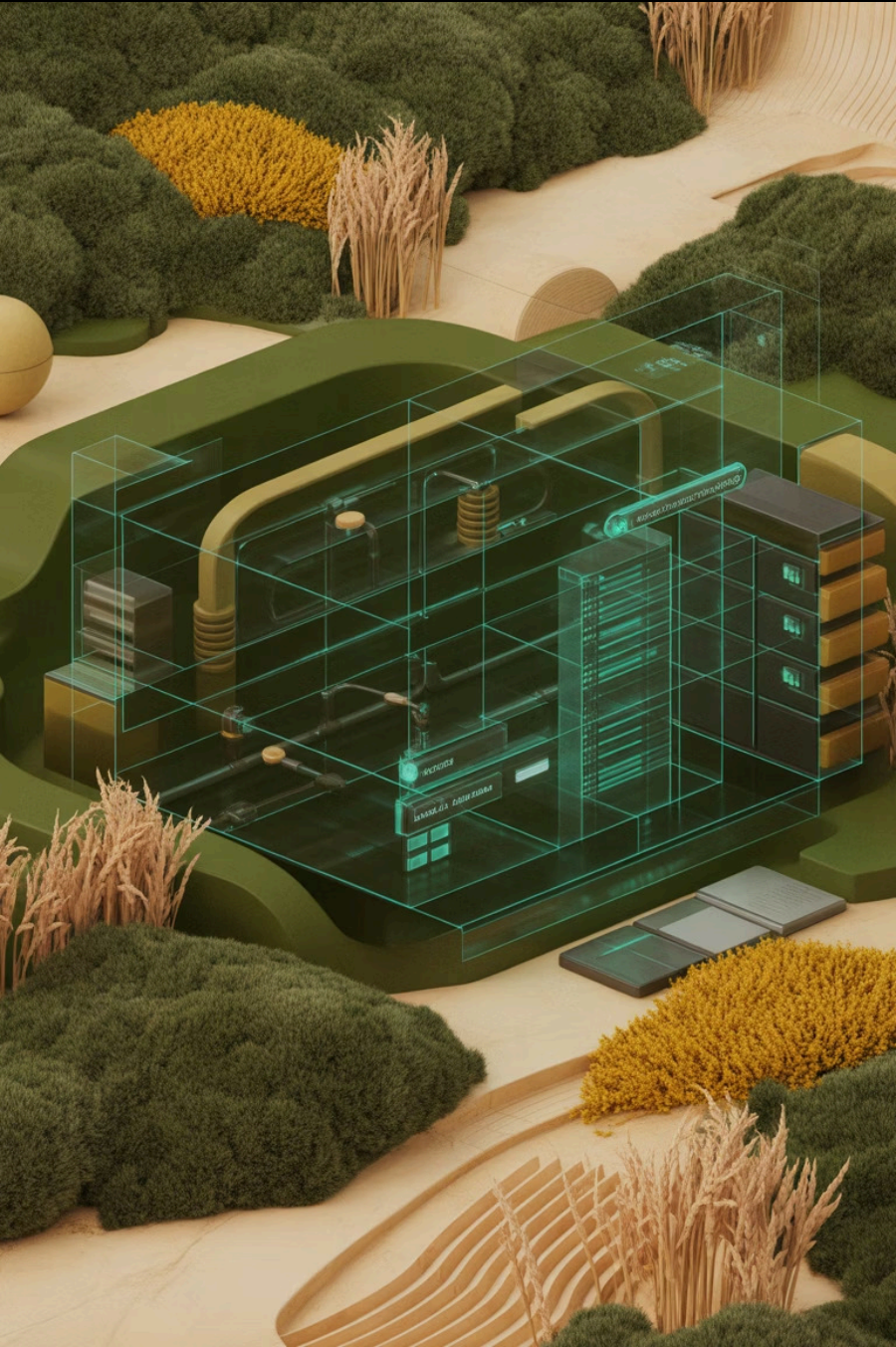
Output

JSON com status codes claros. Erros padronizados: `{ error: string, code?: string }`

Códigos de Status

- 400 - Validação
- 401 - Autenticação
- 403 - Autorização
- 404 - Não encontrado
- 500 - Erro interno

📋 **Critério de sucesso:** Endpoints migrados passam testes unitários e de contrato, mantendo comportamento equivalente ao monólito em tráfego real.



Casos de Borda a Considerar

RBAC

Garantir coerência de permissões entre serviços definindo fonte de verdade clara (Auth/Identity).

Transações Cross-Service

Implementar padrão de sagas e compensações para manter consistência em operações distribuídas.

Dados Duplicados

Estratégias de sincronização e aceitação de eventual consistency entre serviços.

Migração de Dados

Planejar retrocompatibilidade de schema e scripts de migração granular com rollback.

Material Didático

Tópicos de Estudo Essenciais

Arquitetura

- Monólito × Microserviço
- Bounded contexts e DDD
- Trade-offs arquiteturais
- Padrões de decomposição

Dados

- Padrões de migração
- Sagas e compensações
- CDC (Change Data Capture)
- Idempotência

Contratos & Comunicação

- OpenAPI e versionamento
- Testes de contrato
- REST × gRPC × mensagens
- Circuit breaker e retries

Operações

- Logs estruturados
- Métricas e alertas
- Tracing distribuído
- Health checks

Atividades Práticas



Tarefa 1: Especificação

Especificar e documentar API do auth em OpenAPI com todos os endpoints e modelos de dados.



Tarefa 3: Testes

Implementar testes de contrato contra o monólito e validar comportamento equivalente.



Tarefa 2: Scaffold

Criar scaffold de auth-service e mover endpoints de login/refresh mantendo funcionalidade.



Tarefa 4: CI/CD

Dockerizar serviço e criar pipeline CI básico com build, testes e deploy automatizado.

Trabalhar com **PRs pequenos** e incrementais, manter relatório de riscos e decisões por serviço, e demonstrar estratégias de **cutover** e **rollback**.

Scaffold do Auth Service

Estrutura Inicial do Projeto

- 1 package.json**
Scripts (build, start, dev, test, contract:test) e devDependencies (TypeScript/Jest)
- 2 tsconfig.json**
Configuração TypeScript mínima com strict mode e paths configurados
- 3 Dockerfile**
Imagem base otimizada para Node.js com multi-stage build
- 4 README.md**
Guia completo para desenvolvedores: instalação de deps, contrato e ambiente dev
- 5 openapi.yaml**
Contrato com /auth/login e /auth/refresh documentados com exemplos
- 6 test/contract/openapi.spec.ts**
Validação local da especificação OpenAPI
- 7 jest.config.ts**
Configuração do Jest para testes unitários e de integração

Recomendações Práticas

Ações de Baixo Risco Pré-Extração

1

Testes de Contrato

Adicionar testes consumer-driven (Pact) para todas as integrações críticas antes da separação.

2

Shared Library

Versionar shared-lib ou criar monorepo packages/ para itens realmente compartilhados com versionamento semântico.

3

Infraestrutura Mínima

Preparar imagens Docker, health checks, jobs CI e opcionalmente broker de mensagens (RabbitMQ/Kafka).

4

Backup e Migrations

Fazer backup completo das migrations e planejar scripts de migração granular com estratégia de rollback.



Métricas de Sucesso

100%

Cobertura de Testes

Manter cobertura de testes equivalente ou superior ao monólito original

<200...

Latência P95

Garantir que 95% das requisições sejam respondidas em menos de 200ms

99.9%

Disponibilidade

Alcançar SLA de 99.9% de uptime para serviços críticos

0

Incidentes Críticos

Zero incidentes críticos durante o período de cutover gradual